

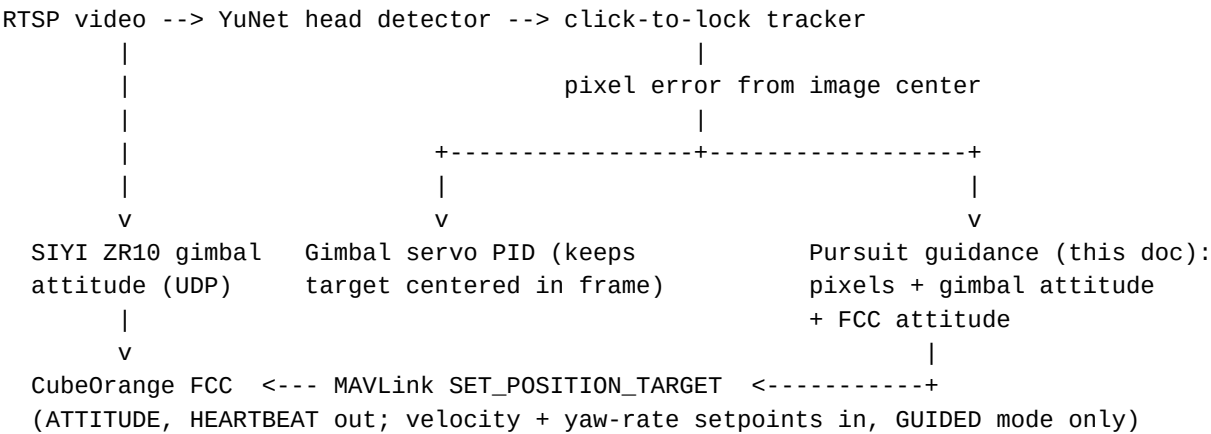
MINI UMWAMBI — Vision-Guided FCC Error Correction

How the gimbal camera, the target detector and the flight controller combine so the UAV can fly nose-on to a visually tracked target, and the mathematics that turn pixels into flight-controller corrections. July 2026.

1. Why this exists

GPS navigation brings the UAV to within a few meters of a destination, but GPS error means 'a few meters off' at the end. This system closes those final meters visually: the ZR10 gimbal camera tracks the target, and the companion computer (Jetson) converts what the camera sees into turn / climb / forward-speed commands for the CubeOrange flight controller (FCC). The UAV points its nose at the target and approaches it head-on. No landing is commanded.

2. The pipeline at a glance



Two control loops share the same pixel measurement. The fast inner loop steers the gimbal so the target stays in view. The outer loop steers the aircraft itself; it must therefore account for how far the gimbal has turned away from the nose, which is where the math below comes in.

3. Step 1 — Pixels to camera angles (pinhole model)

The detector reports the target's box center (cx, cy) in a W×H frame. Its offset from the image center, (px, py) = (cx – W/2, cy – H/2), converts to angles off the camera's optical axis using the pinhole model. The focal lengths in pixels come from the lens field of view (ZR10: HFOV 65°, VFOV 39°):

```
fx = W / (2 * tan(HFOV/2))    fy = H / (2 * tan(VFOV/2))
ax = atan2(px, fx)    (positive = target RIGHT of camera center)
ay = atan2(py, fy)    (positive = target BELOW camera center)
```

This is 'CAMERA CENTER -> TARGET' on the dashboard. Implemented in `offset_to_angles()`, `fcc_landing_feedback.py`.

4. Step 2 — Reference frames and sensor conventions

Three right-handed FRD/NED frames are involved (x forward, y right, z down):

Frame	x axis	Reported by
Camera	out of the lens	pixel angles (Step 1)
Body	out of the UAV nose	target of the correction
World (NED)	true/magnetic north	FCC ATTITUDE message

The two attitude sensors do **not** report in the same frame, and handling this correctly is the heart of the system:

- **FCC (CubeOrange)**: roll, pitch, yaw all earth-referenced; yaw is compass heading.
- **SIYI ZR10**: the gimbal self-stabilizes with its own IMU, so its reported pitch and roll are earth-referenced; but

its yaw (in follow mode) is measured from the UAV nose. A naive subtraction of the two yaw values is therefore meaningless.

5. Step 3 — The mount-relative rotation (camera vs nose)

Every attitude becomes a 3×3 rotation matrix built in the aerospace ZYX order (roll about x, then pitch about y, then yaw about z):

```
R(roll, pitch, yaw) = Rz(yaw) Ry(pitch) Rx(roll)
```

Build the UAV's orientation and the camera's orientation, both relative to the world. The camera's world yaw is the FCC heading plus the gimbal's nose-relative yaw:

```
R_body2world = R(roll_fcc, pitch_fcc, yaw_fcc)
R_cam2world = R(roll_gim, pitch_gim, yaw_fcc + yaw_gim)
```

'Subtracting' orientations is done with matrices, not with the angle values: invert the UAV's rotation (transpose, for rotation matrices) and apply the camera's. What remains is exactly the rotation from the nose to the camera:

```
R_mount = (R_body2world)^T R_cam2world # camera pose in body axes
```

Converting `R_mount` back to angles gives 'UAV NOSE -> CAMERA CENTER' (dashboard row 1 and the bottom bar):

```
yaw = atan2(R21, R11)
pitch = atan2(-R31, sqrt(R32^2 + R33^2))
roll = atan2(R32, R33) # Rij = row i, column j of R_mount
```

Because rotations combine in 3D, these outputs are not plain sums of the inputs. Example: with the camera yawed ~82° and pitched ~19° off the nose, a small UAV pitch shows up mostly as camera *roll* (~3.5° on the dashboard while both raw rolls were near zero) — at 90° of yaw the pitch and roll axes have swapped. Implemented in `siyi_body_from_camera_matrix()` / `mount_relative_euler()`.

6. Step 4 — Nose-to-target bearing (the correction)

Represent the target as a ray in camera axes using the Step-1 angles, rotate it into body axes, and read the bearing off the nose:

```
v_cam = [ 1, tan(ax), tan(ay) ] # pinhole ray, FRD
v_body = R_mount v_cam

azimuth = atan2(v_body.y, v_body.x) # + = target right of nose
elevation = atan2(v_body.z, sqrt(v_body.x^2 + v_body.y^2)) # + = target below nose
```

This pair is 'UAV NOSE -> TARGET', the error the FCC must correct. Note the elevation is measured against the body's horizontal plane, not against the forward axis: `atan2(down, forward)` degenerates as the target approaches 90° of azimuth (the forward component goes to zero and the ratio explodes), which once displayed a fictitious 64° pitch for a target only 19° up. Implemented in `body_frame_angles()`.

7. Step 5 — From error angles to flight commands

The pursuit controller (`fcc_pursuit.py`) converts the bearing into body-frame velocity and yaw-rate setpoints, streamed at 5 Hz:

```
yaw_rate = clamp( 0.8 x azimuth , +/- 45 deg/s ) # swing nose onto target
v_down = clamp( 1.5 x elevation, +/- 1 m/s ) # line up in height
aim_err = max(|azimuth|, |elevation|)
v_fwd = 2 m/s x max(0, 1 - aim_err / 60 deg) # advance only when aimed
```

The forward speed fades linearly to zero as the aim error approaches a 60° cone: the UAV turns in place first, then flies at the target. A camera measures direction, not distance, so the approach speed is constant and the operator (or a future rangefinder) decides when to stop.

The setpoints go out as MAVLink SET_POSITION_TARGET_LOCAL_NED in frame MAV_FRAME_BODY_OFFSET_NED with type_mask 0x05C7 (use vx, vy, vz + yaw rate; ignore position, acceleration and absolute yaw).

8. Safety interlocks

- **Off by default** — the operator arms pursuit with the G key.
- **GUIDED-mode gate** — the companion reads the FCC heartbeat and refuses to send unless the flight mode is GUIDED; flipping the mode switch on the transmitter instantly severs vision's authority.
- **Fail-safe on lock loss** — when the tracker loses the target the companion sends one zero-velocity setpoint and then goes silent; ArduPilot's GUID_TIMEOUT (~3 s) then brakes the vehicle to a hover on its own. A stale command can never keep flying the aircraft.

9. Worked example (real bench data)

From a live dashboard frame: gimbal (19.7, -80.8, 0.0), FCC (-3.1, 71.3, 1.5), target nearly centered in the image (-0.3, +1.3):

Quantity	Pitch	Yaw	Meaning
UAV NOSE -> CAMERA CENTER	+18.7	-82.0	gimbal turned far left, slightly up
+ CAMERA CENTER -> TARGET	-0.3	+1.3	tracker holding target near center
= UAV NOSE -> TARGET	+18.4	-80.6	correction the FCC receives

Reading: the target sits almost dead-center in the video (small row 2), yet the aircraft must yaw ~81° left — because the gimbal, not the aircraft, did the pointing. Sending only the pixel error to the FCC (the naive approach) would have reported 'target centered, no correction needed' while the target was in fact far off the nose. That distinction is the entire reason for Steps 3-4.

10. File map

File	Role
gui_dashboard.py	Dashboard, worker threads, operator controls (lock, G key)
fcc_landing_feedback.py	All frame/rotation math (Steps 1, 3, 4); LANDING_TARGET kept for future use
fcc_pursuit.py	Pursuit control law + MAVLink setpoints + fail-safes (Step 5)
head_tracker.py	YuNet detection, click-to-lock matching, gimbal PID
siyi_zr10.py	SIYI UDP protocol client (attitude, rotate, center)
test_landing_feedback.py / test_pursuit.py	unit tests pinning every convention above